

Reading Benchmarks and Running the System

Inference Fundamentals, Part 3: what every metric is made of, how to rebuild the curves, and why autoscaling is a statement about cold starts

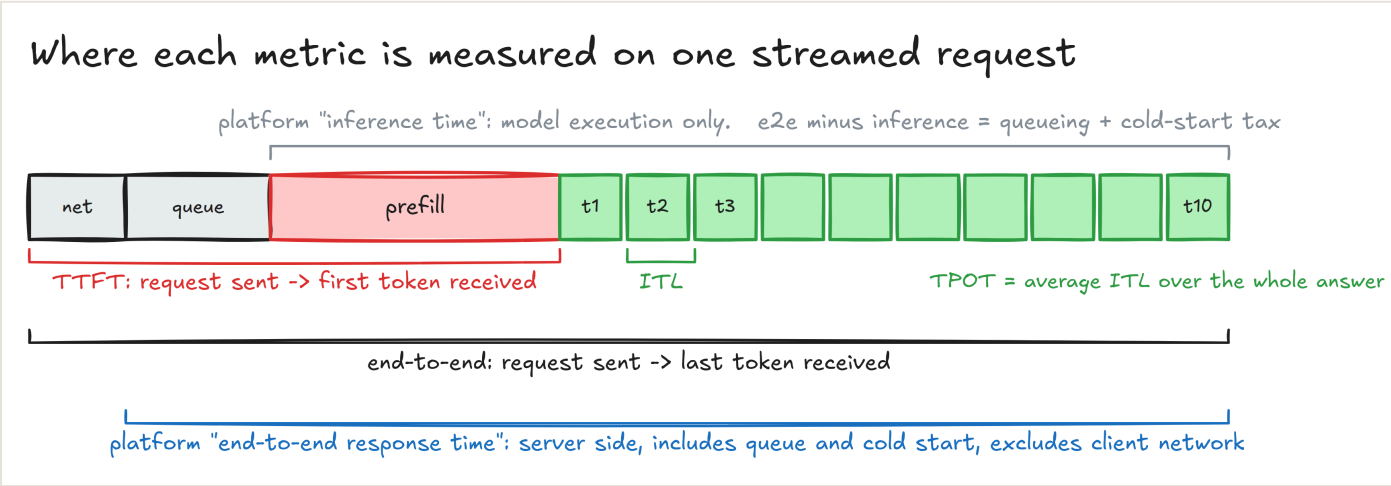
Part 1B covered the physics of a request. Part 2 covered what the serving engine does with it. This part is for the person who has to operate the thing: read a benchmark table without being fooled by it, explain why the 99th percentile misbehaves, convert tokens per second into dollars, and set an autoscaler so that the first user of the morning does not wait a minute. Every formula is followed by what its terms mean, and the numbers are ones you can redo with a pen.

Where does the time go? For more than a decade that was my question, asked of large data platforms on behalf of enterprises: was a query reading, computing, or waiting for a slot; was the compute sized for the concurrency the system actually saw or the concurrency someone guessed; what deserved a cache; and what did all of it cost per query, in terms finance could act on. More recently the question has moved to language models: agentic systems, and the open-source and frontier models beneath them. I came to inference expecting a new set of rules. What I found was the same discipline in different units: two hardware limits, a handful of formulas connecting them to what a user feels, and the familiar problem of which lever a given symptom is asking for. This series is for readers with that background who want to understand inference well enough to run it.

Most conversations about inference performance go wrong in the same place: someone quotes a single number, "3,000 tokens per second," "200 millisecond latency," without saying at what load, at what percentile, or for what prompt length. The point of this article is to make those three questions reflexive, and to give you the small set of formulas that connect the numbers to each other so you can tell when a table is lying.

Anatomy of one request

Every metric is a segment of the same timeline. Here is one streamed request with the segments labeled.



Time to first token (TTFT) runs from the moment the request is sent to the moment the first token arrives. It is made of four things:

TTFT = queue wait + scheduling delay + prefill time + first decode step

Queue wait is time spent waiting for a replica to have a free slot; zero when the system has headroom, unbounded when it does not, and the reason TTFT goes vertical under overload. Scheduling delay is up to one decode step, waiting for the engine's next iteration to admit the request. Prefill time is Part 1B's formula, $\text{input tokens} \times 2 \times \text{parameters} \div \text{FLOPS}$, proportional to prompt length and reduced by prefix caching. The first decode step is one weight read, small next to prefill for any prompt longer than a few hundred tokens.

Inter-token latency (ITL) is the gap between two successive output tokens, which is one decode step:

$$\text{ITL} = \max(\text{weight bytes} + \text{KV bytes}, \text{compute}) \div \text{rate} + \text{overhead}$$

The memory term is all the weights plus every running request's cache, per step, and it grows with $\text{batch} \times \text{context}$. The compute term is $2 \times \text{parameters} \times \text{batch}$ and grows with batch. Whichever is larger sets the step time; that is the knee from Part 1B. **Time per output token (TPOT)** is the same physics averaged across the whole answer: $(\text{end-to-end} - \text{TTFT}) \div (\text{output tokens} - 1)$. Benchmarks report one or the other.

End-to-end latency is what a non-streaming caller waits:

$$\text{E2E} = \text{TTFT} + (\text{output tokens} - 1) \times \text{ITL}$$

For any answer longer than a few dozen tokens this is dominated by $\text{output length} \times \text{ITL}$, which is why capping the maximum output length is the biggest end-to-end lever there is.

Two more vocabulary points matter when reading a serving platform's dashboard. **Inference time** is model execution only, measured inside the replica. **End-to-end response time** is measured at the platform's edge and includes queueing and cold starts. The difference between the two is the queueing and cold-start tax, and it separates "the model is slow" from "the system is overloaded" in one subtraction.

The conversions

These turn engine numbers into customer numbers.

tokens/s per stream	=	$1000 \div \text{ITL (ms)}$	
throughput (tokens/s, replica)	≈	$\text{batch} \div \text{ITL (s)}$	
requests/s	=	$\text{throughput} \div \text{average output tokens}$	
cost per 1M output tokens	=	$(\$/\text{hr} \div 3600) \div \text{throughput} \times 1,000,000$	
concurrent requests	=	$\text{requests/s} \times \text{average E2E (s)}$	(Little's law)

Tokens per second per stream is what one user sees scroll past. Adults read non-fiction silently at about 238 words per minute, about 4 words per second, which is roughly 5 tokens per second; anything above 15 to 20 tokens per second is faster than anyone reads, and an ITL of 24 ms (about 42 tokens per second) is far more than a reader can use. Throughput is the total across all users and is the number that divides into cost. Cost per million tokens is worth doing once by hand: at \$6 per hour, $6 \div 3600 = \$0.001667$ per second; divided by 1,950 tokens per second = \$0.00000855 per token; times a million ≈ \$0.85 per million output tokens. At 3,100 tokens per second it would be about \$0.54, and the 37 percent difference is what the latency columns of a benchmark are buying.

Little's law deserves a sentence on its own. A system holds $\text{arrival rate} \times \text{time-in-system}$ items: 65 requests per second at 3.5 seconds each is about 230 in flight. It converts between the rate a customer quotes and the concurrency an autoscaler needs, and it is a lie detector for tables: if request rate, latency and concurrency do not agree, something is unmeasured, usually retries, cancelled requests, or latency measured at a different point than concurrency.

Which number to trust for utilization

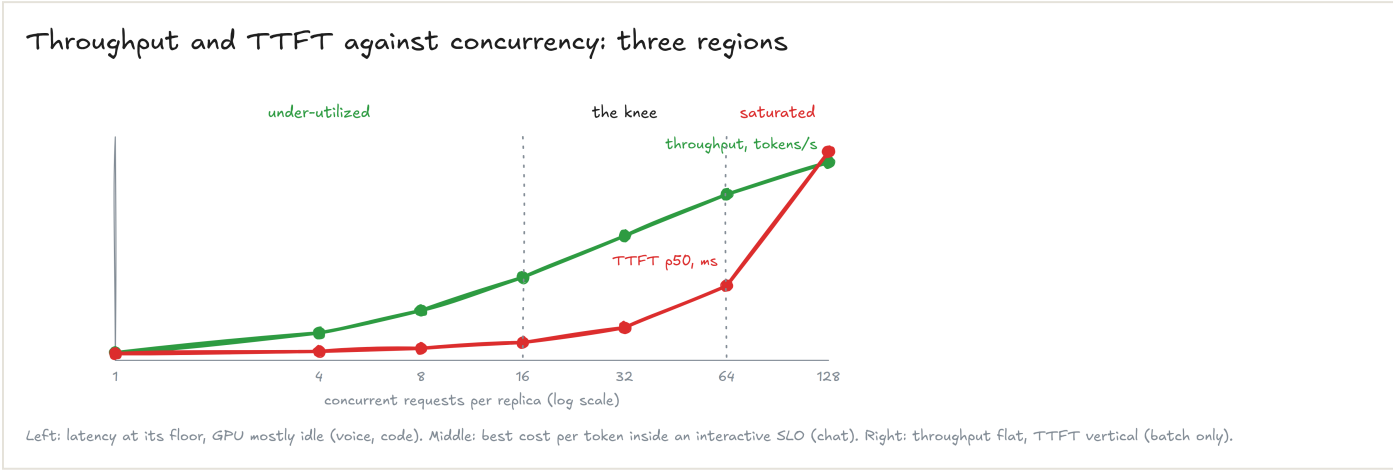
"GPU utilization" in most dashboards means the fraction of time during which any kernel was running. A memory-bound decode step keeps a kernel running while the arithmetic units mostly wait for memory, so utilization can read 95 percent while the GPU does a small fraction of the useful work it could. Tensor core utilization, which measures the share of time the arithmetic units are actually busy, is closer to real compute use. For capacity questions the number that matters is KV cache usage, because it tells you how close the replica is to the memory cliff.

And a word on **goodput**. Throughput counts every token produced. Goodput counts only tokens from requests that met their latency target. Push concurrency past the knee and throughput keeps rising while goodput falls, because a growing share of requests miss the target. When someone argues that a change was good because tokens per second went up, goodput is the number that answers whether the users agreed.

Rebuilding the curves

Take an illustrative benchmark of an 8B-class model on one H100, 1,000 tokens in and 256 out, and rebuild it from Part 1B. If you can explain every row from the physics, no table will surprise you.

CONCURRENCY	ITL MS	TOKENS/S	WHY, FROM THE PHYSICS
1	8	120	Floor is 4.8 ms (16 GB ÷ 3.35 TB/s) plus overhead. One stream at 1000 ÷ 8 = 125.
8	10	780	The same weight read serves 8 streams; compute for 8 tokens still hides under the memory time.
32	16	1,950	KV reads now ≈ 5 GB per step on top of 16 GB of weights; compute for 32 tokens starting to show.
64	24	2,600	Compute per step comparable to memory time. Step time up 50%, throughput up only 33%. The knee.
128	40	3,100	Compute-bound. Doubling batch nearly doubles step time; throughput nearly flat.



TTFT tells a second story. At concurrency 1 it is the prefill of 1,000 tokens (about 16 ms of arithmetic on an 8B) plus overhead, about 45 ms. As concurrency rises, a new request's prefill must be scheduled between other requests' decode steps, and those steps are getting longer, so TTFT creeps up: 60, 80, 120, 220 ms. Then, past the point where the engine's maximum batch is full, new requests wait in a queue for a slot to free up. That wait is not bounded by anything inside the GPU; it depends only on how fast requests arrive versus how fast they finish. That is why TTFT goes vertical while ITL only grows linearly. ITL is physics; queueing is arithmetic.

The three regions on the chart are the practical takeaway. On the left, latency is at its floor and the GPU is mostly idle: right for voice agents and code completion, wrong for the bill. In the middle, the knee, is the best cost per token that still meets an interactive latency target, and it

is where chat products should run. On the right, throughput is flat and every added request makes everyone slower; only batch jobs with no latency target belong there.

Why p99 explodes while p50 barely moves

For the simplest queueing model, a single server with random arrivals and service times (the M/M/1 queue), the expected time a request waits before service depends on utilization ρ , the arrival rate divided by the completion rate:

expected wait $\approx (\rho + (1 - \rho)) \times \text{service time}$

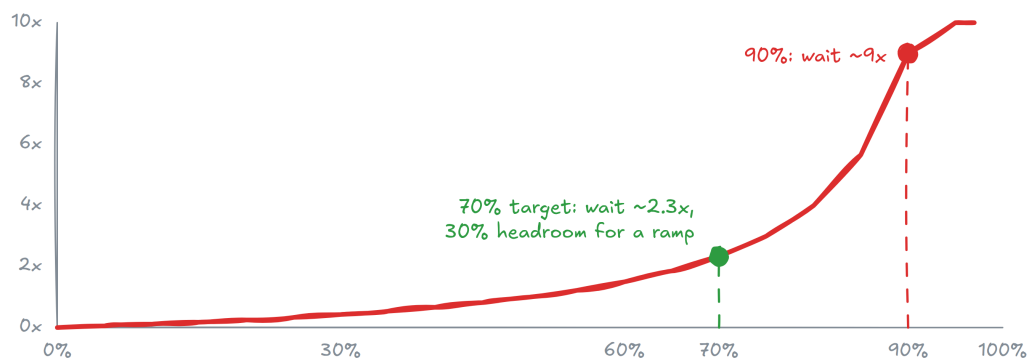
$\rho = 0.50 \rightarrow \text{wait} \approx 1 \times \text{service time}$

$\rho = 0.70 \rightarrow \text{wait} \approx 2.3 \times \text{service time}$

$\rho = 0.90 \rightarrow \text{wait} \approx 9 \times \text{service time}$

$\rho = 0.95 \rightarrow \text{wait} \approx 19 \times \text{service time}$

Queue wait (as a multiple of service time) against utilization



wait $\sim \rho / (1 - \rho)$ service times. Nothing about the server changed along this curve; only the arrival rate did. This is why p99 explodes while p50 barely moves, and why replicas fix queueing while engine tuning does not.

Real serving systems are not exactly M/M/1, but the shape holds: nothing about the server changes along that curve, only the arrival rate does, and from 90 to 95 percent utilization the wait doubles although traffic grew only 5 percent. This single curve explains three things operators see constantly. The median request usually finds a free slot, so p50 stays flat; the 99th-percentile request arrived during a burst and sat in the queue, so p99 rises long before p50 does. A common autoscaling default of about 70 percent target utilization is not conservative for its own sake; the multiplier is still modest there and the remaining 30 percent absorbs traffic that arrives while a new replica is starting. And adding replicas fixes queueing while engine tuning does not: tuning shortens the service time by a few percent, but replicas cut ρ directly and move you left along the curve.

The seven break points

When you are handed a table or a dashboard, there are seven distinct places where behavior changes, each with its own cause and signature. Confusing two of them leads to the wrong fix.

1. **The throughput knee.** Tokens per second gained per added request starts falling. Cause: compute catching up with memory time; K V reads growing. Meaning: diminishing returns on batching.

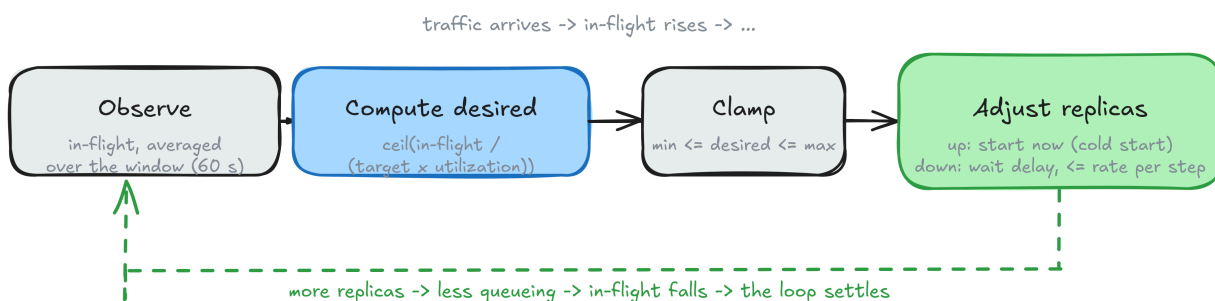
2. **The latency target crossing.** TTFT or ITL at the chosen percentile crosses the customer's line. Not a property of the hardware; the customer's number drawn across the curve. Operate below it with headroom.
3. **The onset of queueing.** TTFT goes vertical; end-to-end diverges from inference time; p99 pulls away from p50. Cause: arrivals exceed completions. Fix: replicas, not tuning.
4. **The memory cliff.** A sharp step, not a bend: latency jumps, throughput drops, restarts climb. Cause: $\text{batch} \times \text{context} \times \text{KV bytes per token}$ exceeded what was left after the weights. Fix: lower concurrency, cap sequence length, quantize or offload the cache, add memory.
5. **The input-length threshold.** TTFT scales with prompt length; ITL jitters when long prompts arrive. Cause: compute-bound prefill. Fix: chunked prefill, prefix caching, a separate deployment for long inputs, disaggregation at scale.
6. **Scaling lag.** In a time series, end-to-end latency spikes at the start of a ramp while desired replicas exceed active ones. Cause: cold start slower than the ramp. Fix: a higher floor, a shorter window, pre-warming.
7. **The capacity ceiling.** Replicas flat at their maximum while in-flight keeps rising. A budget decision, not a technical one.

A benchmark sweep usually shows the first four; a production dashboard usually shows the last three.

The autoscaler as a control loop

An autoscaler is a feedback loop, and it helps to picture it as one. Every few seconds it observes the number of requests in flight, averaged over a window. It computes how many replicas that load needs. It moves the actual count toward that number, subject to a floor, a ceiling, and rules that slow down removal so the system does not thrash.

The autoscaler as a control loop



Example: 230 in flight, target 32, utilization 0.70 -> $32 \times 0.70 = 22.4$ -> $230 / 22.4 = 10.27$ -> 11 replicas desired.
If max replicas = 6: $6 \times 32 = 192$ admitted, 38 queued. Raising max fixes it; tuning the engine does not.

`desired replicas = ceil( in-flight ÷ (concurrency target × target utilization) )`

230 in flight, target 32, utilization 0.70:

$32 \times 0.70 = 22.4$ requests per replica before scaling triggers

$230 \div 22.4 = 10.27 \rightarrow 11$ replicas desired

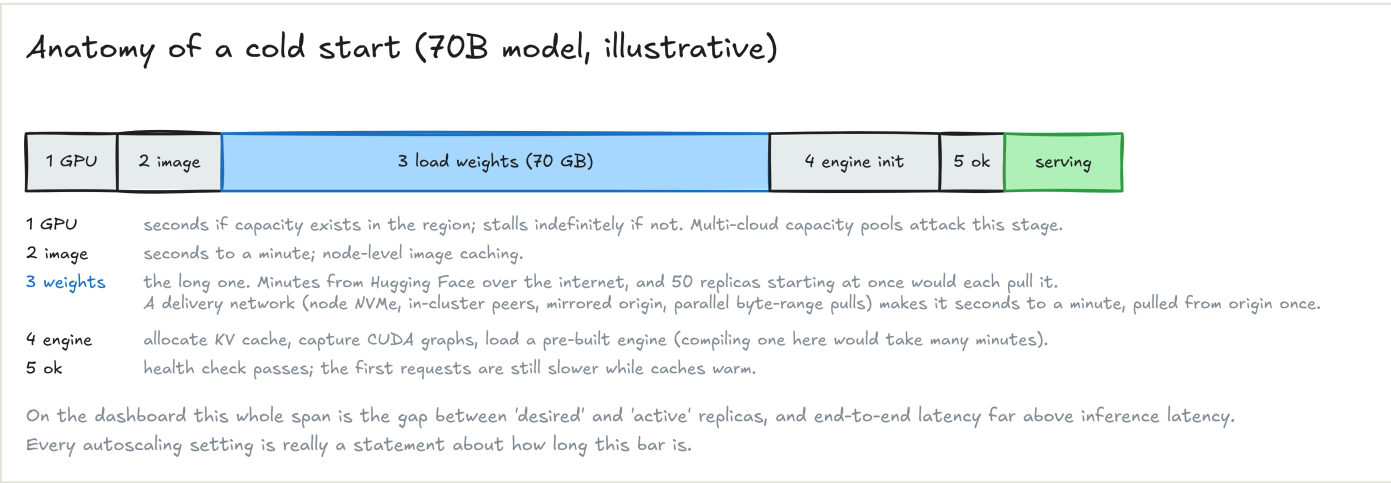
if the maximum is 6: $6 \times 32 = 192$ admitted, 38 queued

Reading the terms. In-flight is requests being served plus requests waiting, averaged over the window; a shorter window reacts faster, a longer one is steadier. Concurrency target is how many requests one replica is configured to handle at once; it should come from a benchmark against the latency target, and per Part 1B it is also a claim that that many requests' KV cache fits. Target utilization is the headroom factor: multiplying the target by 0.7 makes the autoscaler ask for a new replica when replicas are 70 percent full rather than 100, so the spare 30 percent absorbs traffic that arrives during a cold start. Ceil rounds up, because there is no such thing as a fraction of a replica.

Each of the remaining settings exists to fix one specific failure. **The window** smooths noise: too short and a ten-second blip triggers a scale-up you pay for; too long and a real ramp is noticed late. **The scale-down delay** prevents flapping: traffic dips for two minutes, replicas are removed, traffic returns, the system cold-starts again; a delay of ten to fifteen minutes covers most dips. **The maximum scale-down rate** limits how much capacity one step can remove; halving on every quiet minute is aggressive for models whose cold start is measured in minutes. **The minimum replica count** is the floor you pay for so that nobody ever waits for a cold start; size it to the lowest traffic level that still needs sub-second TTFT. **The maximum** is the budget ceiling, and the one setting that turns a performance problem into a business decision.

Anatomy of a cold start

A replica going from nothing to serving passes through five stages, and every autoscaling setting is really a statement about how long they take.



Getting a GPU takes seconds if capacity exists in the region and stalls indefinitely if it does not; multi-cloud capacity pools attack this stage. Pulling the container image takes seconds to a minute with node-level caching. Loading the weights is the long one: a 70B model in FP8 is 70 GB, and DeepSeek-R1 at 671 billion parameters is roughly 700 GB even in FP8. Downloading that from a public model hub over the internet takes minutes and is unreliable, and if fifty replicas start at once they would all download it separately. Managed platforms solve this with a delivery network that mirrors weights at deploy time and pulls them through tiers of cache (node-local NVMe, in-cluster peers, a mirrored origin with parallel byte-range fetches), so a herd of replicas pulls the model from origin once. Engine initialization allocates the KV cache, captures CUDA graphs, and loads a pre-built engine; compiling one on the spot would take many minutes. Then the health check passes, and the first few requests are still slower while caches warm.

On a dashboard this whole span is the gap between desired and active replicas, and end-to-end latency far above inference latency. That signature is unambiguous once you know to look for it.

If a replica took one second to start, you would run the floor at zero, a ten-second window, and never think about autoscaling again. The longer the cold start, the more headroom you need, the longer the scale-down delay should be, and the higher the floor has to sit. So when someone says "scale-to-zero will save us money," the reply is a question: what is the cold start on this model, and what does the first user of the morning see?

A day on the dashboard

Here is what all of this looks like on a real deployment. Illustrative numbers for a chat product configured with concurrency target 32, utilization 70 percent, minimum replicas 0, maximum 6. Response times in seconds, end-to-end, non-streaming.

HOUR	REQ/S	CONCURRENT	ACTIVE REPLICAS	E2E P50	E2E P99	INFERENCE P99	WHAT IS HAPPENING
05:00	0	0	0				Scaled to zero overnight
06:00	3	36	0 → 1	12.0	18.5	3.8	Scaling lag. E2E is 15 s above inference: a cold start, not the model
08:00	20	60	2	2.8	4.1	3.9	Healthy. E2E $\approx$ inference. Little's law: $20 \times 2.8 \approx 56$
11:00	65	230	6 (max)	3.5	9.5	4.0	Capacity ceiling plus queueing . 230 in flight vs 192 admissible; the autoscaler wants 11
13:00	38	110	6	2.9	4.2	3.9	Recovered. Still 6 replicas: the scale-down delay holding capacity, correctly
22:00	2	5	1	2.7	4.0	3.8	Single replica, fine

Inference p99 is flat at about 4 seconds all day, so the model is never the problem. The 06:00 incident is a cold start: three requests per second piling up for twelve seconds while the first replica loads. The 11:00 incident is queueing: 230 in flight against $6 \times 32 = 192$ admissible, and the autoscaler wanted $\text{ceil}(230 \div 22.4) = 11$ replicas but was capped at 6. Both have one-line fixes with a cost attached: a floor of one replica on weekdays removes the morning incident for the price of one idle GPU overnight; a ceiling of twelve removes the lunchtime incident for the price of up to six extra GPUs for about two hours a day. The 13:00 row is the autoscaler behaving correctly, holding capacity through the post-lunch dip. And the next thing to ask for is request size p99 by hour, because if the 11:00 crowd also sends longer prompts, part of that 9.5 seconds is prefill and adding replicas will help less than claimed.

Check yourself

- 1. p99 TTFT is 4 seconds and p50 is 300 milliseconds. Model problem or capacity problem?** Capacity. The model serves the median request in 300 ms. The slow requests queued, hit a cold start, or carried unusually long prompts. Confirm with end-to-end minus inference time, desired versus active replicas, and request size p99.
- 2. 230 in flight, target 32, utilization 70 percent, maximum 6 replicas. What happens?** Desired = $\text{ceil}(230 \div 22.4) = 11$, capped at 6. 192 admitted, 38 queued. End-to-end climbs while inference stays flat. Raising the maximum fixes it; tuning the engine does not.
- 3. A benchmark shows 3,100 tokens per second at concurrency 128. Should an interactive product run there?** No. TTFT and ITL are several times what they were at 32, and goodput for a chat product is probably lower even though throughput is higher. A batch job, yes.
- 4. Why is "GPU utilization 95 percent" not proof a replica is at capacity?** It measures time a kernel was running, not useful work. A memory-bound decode step shows high utilization while the arithmetic units idle. Tensor core utilization and, above all, KV cache usage are the honest capacity signals.
- 5. Convert: \$6 per hour, 1,950 tokens per second. Cost per million output tokens?** $6 \div 3600 = \$0.001667$ per second; $\div 1,950 = \$0.000000855$ per token; $\times 1,000,000 \approx \0.85 .

Closing

Four articles so far, one idea: an LLM request is bound by memory bandwidth in one phase and by arithmetic in the other, and everything else, from the shape of a benchmark curve to the right value of an autoscaling setting, follows from that. If you take one habit from the series, make it the reflex to ask three questions of any performance number: at what load, at what percentile, and for what prompt length. Part 4 applies the same idea to the decisions that sit on top of the operator's view: quantization, structured outputs, tool calling, and the tail latency of agents.

Glossary

Activations. Intermediate values produced while a forward pass runs; short-lived, but they take GPU memory alongside weights and KV cache. **All-reduce.** The exchange of partial results between GPUs after each layer in tensor parallelism. Cheap over NVLink, expensive over a network. **Batch size.** How many sequences are processed together in one step; in decode, the number of concurrent requests. **Chunked prefill.** Splitting a long prompt's prefill into pieces interleaved with other requests' decode steps. **Concurrency.** Requests in flight on a replica at once; the x-axis of every benchmark. **Continuous batching.** Scheduling where requests join and leave the batch at every step rather than waiting for a whole batch to finish. **Decode.** The phase that generates output one token per step. Memory-bandwidth-bound. **Disaggregated serving.** Prefill and decode on separate GPU pools with the KV cache shipped between them. **Draft model.** The small model in speculative decoding that proposes tokens for the target to verify. **Expert parallelism.** Placing different experts of a mixture-of-experts model on different GPUs. **FLOPS.** Floating-point operations per second, the arithmetic limit. Prefill is bound by it. **FP8, FP4.** 8-bit and 4-bit floating-point formats; native on H100 and B200 respectively. **Goodput.** Throughput counting only requests that met their latency target. **GQA.** Grouped-query attention: fewer KV heads than query heads, cutting the KV cache several-fold. **HBM.** High-bandwidth memory on the GPU; its size bounds what fits, its bandwidth bounds decode speed. **In-flight.** Requests accepted but not completed, including those waiting in queue. **ITL / TPOT.** Inter-token latency / time per output token; one decode step. **KV cache.** Per-request store of attention keys and values for every token processed so far. **Little's law.** Items in a system = arrival rate $\times$ time in system. **Memory-bound / compute-bound.** Which GPU limit a workload hits first: bandwidth (decode) or FLOPS (prefill). **MoE.** Mixture of experts: many sub-networks per layer, few active per token. Large memory, smaller per-token compute. **PagedAttention.** Block-based KV cache allocation that eliminates fragmentation and enables prefix sharing. **Preemption.** Evicting a running request's KV cache when memory runs out; the inside view of the memory cliff. **Prefill.** Processing the whole prompt in one pass. Compute-bound. Produces the first token and the initial KV cache. **Prefix caching.** Reusing KV cache for a shared prompt prefix across requests. **Quantization.** Storing weights, and optionally the KV cache, in fewer bits. **Replica.** One running copy of the model server, on one or more GPUs; the unit the autoscaler adds and removes. **Speculative decoding.** Proposing several tokens cheaply and verifying them in one target-model step. **Tensor parallelism.** Splitting each layer's weights across GPUs. **Throughput.** Tokens per second across all requests on a replica. **TTFT.** Time to first token: queue wait plus prefill, essentially.

Illustrative benchmark and dashboard numbers are constructed to have the shape and internal consistency of real data; run your own before relying on any of them.

Radx Radhakrishnan works on LLM inference and agentic systems, after a decade of performance work on enterprise data platforms.

Sources

- NVIDIA, H100 Tensor Core GPU (80 GB, 3.35 TB/s; used in the rebuilt benchmark). <https://www.nvidia.com/en-us/data-center/h100/>
- Brysbaert, "How many words do we read per minute? A review and meta-analysis of reading rate," Journal of Memory and Language, 2019 (238 wpm non-fiction, 260 wpm fiction). <https://www.sciencedirect.com/science/article/abs/pii/S0749596X19300786>
- OpenAI Help Center, "What are tokens and how to count them?" (≈ 0.75 words per token). <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>
- Little's law ($L = \lambda W$). https://en.wikipedia.org/wiki/Little%27s_law
- M/M/1 queue, mean waiting time in queue $W_q = \rho / (\mu(1 - \rho))$. https://en.wikipedia.org/wiki/M/M/1_queue
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023 (preemption and KV cache behavior under memory pressure). <https://arxiv.org/abs/2309.06180>
- NVIDIA, "Introducing NVIDIA Dynamo," March 18, 2025 (KV-aware routing and disaggregation in production serving). <https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/>
- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024 (671B parameters, the basis of the weight-size example). <https://arxiv.org/abs/2412.19437>